# Secure E-Learning Platform: Design, Implementation and Testing

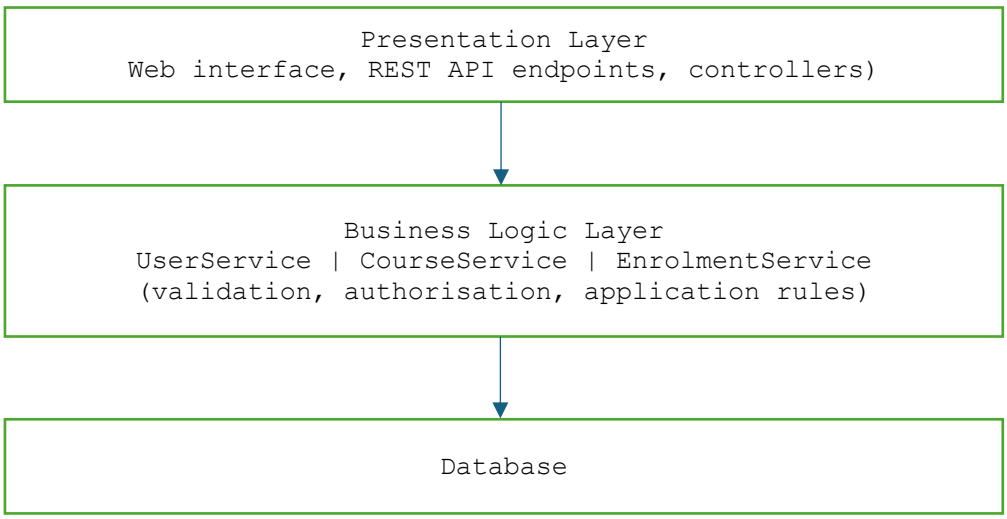
## Task 1: Software Architecture Design

### Chosen architecture style

I chose a layered architecture for this platform, made up of a presentation layer, a business logic layer and a data access layer. I did think about microservices as an option, but for a system this size, a single person building a coursework sized e-learning platform, I felt microservices would bring in a level of complexity, service discovery, communication between services, managing data across multiple services, that I did not need to take on for what this assignment actually required. This lines up with something Tibi (2022) talks about in relation to test driven development, that simpler, well structured designs are easier to test on their own and easier to reason about, which mattered to me since Task 2 asked for the module to be built test first from the start.

I still think a layered structure supports scalability and maintainability, because each layer has one job and can be changed without dragging the other layers along with it. If this platform ever needed to scale properly, I could pull individual layers out into separate services later without redoing the whole design, so going layered now does not shut that door, it just means I am not taking on complexity I do not need yet.

### High level architecture diagram




Requests move down through these layers and responses come back up, with each layer only talking to the one directly below it. I kept it this way so responsibilities

stay separated and so I could test the business logic on its own, without needing a working interface or a real database in place, which mattered once I got into the test driven part of this assignment.

**Key modules**

I identified four core modules for the platform.

- User Management, handling registration, authentication and role assignment, for example student, instructor or administrator.
- Course Management, handling creating, updating, listing and removing courses. This is the module I built out in full for this assignment.
- Enrolment Management, handling linking students to courses and tracking enrolment status.
- Assessment Management, handling assignments, submissions and grading, which I have left out of scope for this assignment but would sit alongside the other modules in a complete build.

**Security considerations**

I thought about security at each layer rather than treating it as one extra feature added at the end.

At the presentation layer, I planned for all input coming from users, whether through a web form or an API request, to be validated before it reaches the business logic, to cut down the risk of malformed or malicious input getting further into the system. At the business logic layer, I decided authentication should use hashed passwords rather than storing anything in plaintext, using something like bcrypt, and that role based access control should be in place so that, for example, only instructors or administrators can create or change courses rather than any logged in user. At the data access layer, I planned to use parameterised queries instead of building queries by concatenating strings, to avoid injection attacks. Data moving between the client and the platform would be encrypted over HTTPS in a real deployment, and I made sure sensitive fields such as password hashes are never something I would log or expose in error messages.

If this platform ever grew to include AI driven features, something like personalised course recommendations, I would need to think about responsible AI principles as part of that design too, since Lu et al. (2022, 2024) argue that governance and fairness need to be built in from the design stage rather than bolted on once an AI feature already exists. I have not built anything like that here, but it is something I would want to keep in mind if the platform moved in that direction.

## Task 2: Implementation of the Course Management Module

### Test driven development approach

I followed a test driven development approach for this module, writing the unit tests first and only writing enough implementation code to make each test pass before moving on to the next piece of behaviour. This is the approach Tibi (2022) describes, where the tests are written against the intended behaviour before that behaviour exists, so the tests end up acting as a kind of specification as well as a check. AGBLA (2024) makes a similar point about Angular development, that writing the test first forces you to think clearly about expected inputs and outputs before getting caught up in how you are actually going to build it, and I found that to be true while working through this, since it pushed me to think about things like duplicate course codes before I had written any logic to deal with them.

I used Python's built in unittest module for this.

### Step 1: writing the tests first

I wrote the tests below before any implementation code existed.

python

```
import unittest
from course_management import CourseManager, Course, DuplicateCourseError, CourseNotFoundError


class TestCourseManager(unittest.TestCase):

    def setUp(self):
        self.manager = CourseManager()

    def test_add_course_successfully(self):
        course = self.manager.add_course("CS101", "Introduction to Programming", 15)
        self.assertEqual(course.code, "CS101")
        self.assertEqual(course.title, "Introduction to Programming")
        self.assertEqual(course.credits, 15)

    def test_add_course_with_duplicate_code_raises_error(self):
        self.manager.add_course("CS101", "Introduction to Programming", 15)
        with self.assertRaises(DuplicateCourseError):
            self.manager.add_course("CS101", "Another Course", 10)

    def test_add_course_with_empty_title_raises_error(self):
        with self.assertRaises(ValueError):
            self.manager.add_course("CS102", "", 15)

    def test_add_course_with_invalid_credits_raises_error(self):
        with self.assertRaises(ValueError):
            self.manager.add_course("CS103", "Data Structures", -5)

    def test_get_course_by_code(self):
        self.manager.add_course("CS101", "Introduction to Programming", 15)
        course = self.manager.get_course("CS101")
        self.assertEqual(course.title, "Introduction to Programming")
```

```
    def test_get_course_that_does_not_exist_raises_error(self):
        with self.assertRaises(CourseNotFoundError):
            self.manager.get_course("CS999")

    def test_update_course_title(self):
        self.manager.add_course("CS101", "Introduction to Programming", 15)
        updated = self.manager.update_course("CS101", title="Programming Fundamentals")
        self.assertEqual(updated.title, "Programming Fundamentals")

    def test_remove_course(self):
        self.manager.add_course("CS101", "Introduction to Programming", 15)
        self.manager.remove_course("CS101")
        with self.assertRaises(CourseNotFoundError):
            self.manager.get_course("CS101")

    def test_list_courses_returns_all_added_courses(self):
        self.manager.add_course("CS101", "Introduction to Programming", 15)
        self.manager.add_course("CS102", "Data Structures", 20)
        courses = self.manager.list_courses()
        self.assertEqual(len(courses), 2)


if __name__ == "__main__":
    unittest.main()
```

When I ran this file at this point, it failed straight away, since course_management.py did not exist yet. That was expected, it is just the starting point of working test first, where the tests fail before any of the actual code is written.

### Step 2: writing the implementation

Once I had the tests above in place, I wrote the implementation below, trying to write only what I needed to get each test passing.

python

```
class Course:
    def __init__(self, code, title, credits):
        self.code = code
        self.title = title
        self.credits = credits

    def __repr__(self):
        return f"Course({self.code}, {self.title}, {self.credits} credits)"


class DuplicateCourseError(Exception):
    pass


class CourseNotFoundError(Exception):
    pass


class CourseManager:
    def __init__(self):
```

```
        self._courses = {}

    def add_course(self, code, title, credits):
        if not code or not title:
            raise ValueError("Course code and title cannot be empty.")
        if credits <= 0:
            raise ValueError("Credits must be a positive number.")
        if code in self._courses:
            raise DuplicateCourseError(f"A course with code {code} already exists.")

        course = Course(code, title, credits)
        self._courses[code] = course
        return course

    def get_course(self, code):
        course = self._courses.get(code)
        if course is None:
            raise CourseNotFoundError(f"No course found with code {code}.")
        return course

    def update_course(self, code, title=None, credits=None):
        course = self.get_course(code)
        if title is not None:
            if not title:
                raise ValueError("Course title cannot be empty.")
            course.title = title
        if credits is not None:
            if credits <= 0:
                raise ValueError("Credits must be a positive number.")
            course.credits = credits
        return course

    def remove_course(self, code):
        self.get_course(code)
        del self._courses[code]

    def list_courses(self):
        return list(self._courses.values())
```

Once this was in place, all nine tests passed when I ran the test file with `python3 -m unittest test_course_management.py`.

## Task 3: Testing, Validation and Refactoring

### Test coverage

I wrote tests covering every public method on the CourseManager class, including the normal success paths and the main ways things could go wrong, duplicate course codes, empty titles and invalid credit values. I used setUp to create a fresh CourseManager before every test, so nothing carried over between tests and each one ran on its own.

I ran the full test suite locally in the terminal using `python3 -m unittest test_course_management.py`, and all nine tests passed on the first run after I refactored the code. The output is shown below.

```
.........
----------------------------------------------------------------------
Ran 9 tests in 0.002s

OK
```

A screenshot of this output is included in the appendix as evidence of the test run.

### Refactoring

Once I had confirmed all the tests passed, I went back through the implementation to see where I could tidy it up. I noticed I had written the same credit validation check, making sure credits was greater than zero, in both add_course and update_course. I pulled that out into a single helper method instead, shown below, and reran the full test suite afterwards to make sure nothing had changed in how the code actually behaved.

python

```python
class CourseManager:
    def __init__(self):
        self._courses = {}

    def _validate_credits(self, credits):
        if credits <= 0:
            raise ValueError("Credits must be a positive number.")

    def add_course(self, code, title, credits):
        if not code or not title:
            raise ValueError("Course code and title cannot be empty.")
        self._validate_credits(credits)
        if code in self._courses:
            raise DuplicateCourseError(f"A course with code {code} already exists.")

        course = Course(code, title, credits)
        self._courses[code] = course
        return course

    def get_course(self, code):
        course = self._courses.get(code)
```

```
        if course is None:
            raise CourseNotFoundError(f"No course found with code {code}.")
        return course

    def update_course(self, code, title=None, credits=None):
        course = self.get_course(code)
        if title is not None:
            if not title:
                raise ValueError("Course title cannot be empty.")
            course.title = title
        if credits is not None:
            self._validate_credits(credits)
            course.credits = credits
        return course

    def remove_course(self, code):
        self.get_course(code)
        del self._courses[code]

    def list_courses(self):
        return list(self._courses.values())
```

Running the same nine tests against this version confirmed everything still passed, which is one of the things I took away from this exercise, that having a solid set of tests in place meant I could restructure the code afterwards with confidence, since anything I broke while refactoring would have shown up straight away in a failing test rather than going unnoticed. AGBLA (2024) makes this same point about the value of a test suite once you start reworking code.

**Discussion of Design Decisions**

I went with a layered architecture mainly because of the scale of this assignment and because I wanted the design to stay approachable while still showing clear separation between the different parts of the system, which fits with what Tibi (2022) says about designing systems that are easy to test in isolation. Writing the tests before writing the code, as Task 2 asked for, changed how I worked through this. I had to think through what CourseManager was actually supposed to do, including how it should fail, before I had written any of the logic itself, rather than writing the code first and testing it as an afterthought.

The security considerations I laid out in Task 1 are only partly reflected in what I actually built for Course Management, since this module does not handle authentication or encryption directly. In a fuller build, the CourseService layer sitting above CourseManager would be the place I would check that whoever is calling these methods actually has the right role, instructor or administrator, before letting them call something like add_course or remove_course, which is the role based access control approach I described earlier.

**References**

AGBLA, E.A. (2024) *Mastering Angular test-driven development: build high quality Angular apps with step-by-step instructions and practical examples*. Birmingham: Packt Publishing.

Lu, Q. et al. (2022) *Responsible AI by design: a pattern collection for designing responsible AI systems*. arXiv.org.

Lu, Q. et al. (2024) 'Responsible AI pattern catalogue: a collection of best practices for AI governance and engineering', *ACM Computing Surveys*, 56(7).

Tibi, A. (2022) *Pragmatic test-driven development in C# and .NET: write loosely coupled, documented, and high quality code with DDD using familiar tools and libraries*. Birmingham: Packt Publishing.